

When Does Context Compression Help? Multi-Turn Mathematical Reasoning with Small Language Models

Anonymous ACL submission

Abstract

Context compression is increasingly proposed as a remedy for multi-turn reasoning degradation, yet when it helps remains underexplored. We investigate this question for small language models (SLMs) on multi-turn mathematical reasoning. We introduce *Sparse Logic State Compression* (SLSC), a training-free twin-agent framework that maintains reasoning state as atomic mathematical propositions, and evaluate it on Math-100-multi, a 100-problem benchmark with explicit numerical dependencies. Three findings emerge: compression substantially benefits models at 4B parameters and above but yields no consistent gain at 2B or below, revealing a capability threshold around 4B; natural-language summary matches or exceeds structured SLSC in accuracy, suggesting the primary mechanism is bounded context size rather than typed representation; and formal type augmentation consistently degrades accuracy across all scales—by up to 5 percentage points—in the absence of verifier feedback. These findings clarify when compression helps and offer practical guidance for deploying SLMs in multi-turn settings.

1 Introduction

Multi-turn interaction is rapidly becoming the dominant deployment mode for language models, yet recent work documents substantial performance degradation: (Laban et al., 2025) report an average 39% accuracy drop across six generation tasks over 200,000 conversations with 15 frontier LLMs. We study a specific instance of this phenomenon: *multi-turn mathematical reasoning with explicit numerical dependencies*, where each sub-question depends on the numerical result of the preceding turn. This setting is particularly acute for small language models (SLMs, $\sim 1\text{B}$ – 10B parameters), whose limited attention capacity makes them vulnerable to compounding errors across turns.

The central question is how to maintain mathematical intermediate state across turns. Current approaches store state as unstructured text: prompt compression methods like LLMingua (Jiang et al., 2023b) operate at the token level, agnostic to mathematical structure; reasoning-summarization methods like Reflexion (Shinn et al., 2023) and InfyThink (Yan et al., 2025) condense reasoning into natural-language narratives where numerical constraints receive no privileged representation. Raw context accumulation defers the problem until attention can no longer locate relevant constraints (Liu et al., 2024). We argue these approaches assume reasoning state should be flat text—a poor match for mathematical reasoning, where state is intrinsically typed: invariants, active variables, binding equations, each with distinct roles in subsequent inference.

We propose *Sparse Logic State Compression* (SLSC), a twin-agent framework where a reasoning agent $\mathcal{A}$ generates natural-language chains of thought while a curator agent $\mathcal{B}$ distills accumulated context into atomic mathematical propositions. The reasoning agent’s context is replaced by this structured state, giving each constraint a persistent, addressable location. Figure 1 contrasts the two representations.

We evaluate SLSC on Math-100-multi, a 100-problem multi-turn benchmark derived from MATH-500 (Hendrycks et al., 2021) (L3–L5), where each problem decomposes into 2–5 dependent sub-questions. We compare four strategies across four Qwen3.5 reasoner scales (Qwen Team, 2026) (0.8B–9B). At 4B and above, compression methods substantially outperform raw history accumulation, with SLSC achieving strong gains while reducing per-turn tokens by 62%. However, augmenting SLSC with Lean 4 formalization consistently degrades accuracy, suggesting stateless auto-formalization without verifier feedback introduces noise.

Baseline: Natural-Language Summary	SLSC: Sparse Logic State
<i>Flat text; intermediate constraints embedded in prose.</i> Context passed to Agent $\mathcal{A}$ at Turn 3: “To approach this problem, I first considered the divisors of $n + 7$ and $2n + 1$. By examining the linear combination $2(n + 7) - (2n + 1)$, I observed that this simplifies to 13, which means any common divisor d must divide 13. Since 13 is prime, the possible values are $d = 1$ or $d = 13$. I then proceeded to analyze the case $d = 13$, which requires $n + 7 \equiv 0 \pmod{13}$, leading to $n \equiv 6 \pmod{13}$. Combined with the bounds, I narrowed down to candidates $n = 6, 19, 32, \dots$, verifying each against the secondary constraint ...” ~185 tokens • no addressable structure • key fact mid-sentence	<i>Atomic propositions; each independently addressable.</i> Context passed to Agent $\mathcal{A}$ at Turn 3: 1. d divides $n + 7$ 2. d divides $2n + 1$ 3. $2(n + 7) - (2n + 1) = 13$ 4. d divides 13 5. $d \in \{1, 13\}$ 6. If $d = 13$: $n \equiv 6 \pmod{13}$ 7. Candidates: $n \in \{6, 19, 32, \dots\}$ ~55 tokens • each fact retrievable • persistent slot per constraint

Figure 1: Comparison of context representations at Turn 3 of a multi-turn mathematical reasoning task. **Left:** Natural-language summary embeds constraints in prose, requiring attention over unstructured text to locate key facts (highlighted). **Right:** SLSC maintains atomic propositions in numbered slots, giving each constraint a persistent and addressable location. SLSC reduces token count by $\sim 70\%$ while improving constraint retrievability.

We make three contributions:

1. **Dataset.** Math-100-multi, a 100-problem multi-turn benchmark with explicit numerical dependencies, derived from MATH-500 (L3–L5).
2. **Structured state compression improves accuracy at 4B+ scales.** SLSC achieves substantial accuracy gains over raw history accumulation at 4B and above, while reducing per-turn input tokens by 62%. Natural-language summary shows comparable or larger gains, suggesting benefits come from bounded context size rather than structured representation alone.
3. **Cautionary finding on formal augmentation.** Lean 4 type stubs consistently degrade accuracy across all scales, attributed to stateless autoformalization without verifier feedback.

2 Related work

Multi-turn reasoning degradation. Recent work documents systematic performance loss in multi-turn settings. Laban et al. (2025) report an average 39% accuracy drop across six generation tasks over 200,000 simulated conversations with 15 frontier models, characterizing failure as a “lost-in-conversation” phenomenon. Liu et al. (2024) show that language models do not robustly use information in long contexts, with retrieval accuracy varying sharply by position. Kwan et al. (2024)

benchmark multi-turn capabilities across recollection, expansion, refinement, and follow-up interactions, finding that error propagation across turns is a primary driver of performance loss. Zheng et al. (2023) introduce MT-Bench as an 80-question multi-turn evaluation framework with LLM-as-judge scoring, establishing a standard protocol for multi-turn dialogue assessment. These findings establish multi-turn degradation as broad-spectrum, but existing multi-turn benchmarks largely target underspecification and clarification rather than numerical chaining. Our dataset complements this literature by isolating numerical-dependency failures, where each sub-question is fully specified but its solution requires the prior turn’s numerical result.

Prompt and token-level compression Prompt compression methods reduce input length while attempting to preserve task performance. LLMLingua (Jiang et al., 2023b) and its extensions (Jiang et al., 2024) identify and remove low-information tokens via small auxiliary models, operating at the lexical level without awareness of which tokens encode task-critical structure. KV-cache eviction methods such as H2O (Zhang et al., 2023) and SnapKV (Li et al., 2024) selectively retain high-attention tokens during generation, while StreamingLLM (Xiao et al., 2024) identifies attention sinks at initial positions that receive disproportionate weight regardless of semantic content. By design, a numerical constant inside an inequal-

ity and a discourse adjective receive comparable treatment under token-level methods. SLSC differs along two axes: compression operates over semantic units—atomic mathematical propositions—rather than tokens, and is performed by a dedicated curator agent prompted to identify mathematical content. We view the two approaches as orthogonal and do not include token-level compression in our experimental comparison, which focuses on state-maintenance strategies sharing the same multi-turn decomposed setup.

Reasoning-state compression and summarization A closer line of work compresses intermediate reasoning state rather than raw inputs. Reflexion (Shinn et al., 2023) condenses prior trajectories into natural-language reflections that condition subsequent attempts. InftyThink (Yan et al., 2025) extends this idea to long-horizon reasoning by iteratively summarizing chain-of-thought and re-prompt the model on the summary, achieving large context savings. Conceptually these methods sit closest to SLSC, and two differences are central. First, both produce flat natural-language summaries in which numerical constraints stated in passing receive no privileged representation; SLSC enforces an atomic-proposition schema in which each targets monolithic single-question reasoning with internally-induced summarization points and assumes a trained summarization checkpoint, whereas our setting is externally-decomposed multi-turn dialogue with explicit numerical dependencies and is fully training-free. A head-to-head empirical comparison would require either retraining InftyThink on our dependency-annotated dataset or rebuilding it as a prompted pipeline; we leave this to future work and restrict our experiments to training-free, setup-compatible baselines.

Implicit assumptions in compression literature

The compression methods above share an implicit assumption: that reducing context length improves downstream task performance, either by mitigating attention dilution (Liu et al., 2024) or by enabling longer reasoning chains within fixed context budgets. This assumption is rarely tested in controlled settings where compressed and uncompressed strategies are compared on identical problems with statistical rigor. Our work directly evaluates this assumption in the multi-turn mathematical reasoning regime. We find that compression methods substantially outperform raw history accu-

mulation in accuracy at 4B parameters and above (Section 5), while delivering substantial per-turn token savings. This positions compression as both an accuracy intervention and a serving-cost intervention, and suggests that prior claims of compression-driven accuracy gains are well-founded in this regime, though the benefit comes primarily from bounded context size rather than from structured representation per se.

Formal language augmentation for reasoning.

A separate line of work explores formal-language augmentation for mathematical reasoning. Auto-formalization (Wu et al., 2022) translates natural-language statements into formal expressions; Draft, Sketch, and Prove (Jiang et al., 2023a) interleaves natural-language drafting with formal verification; LeanDojo (Yang et al., 2023) provides infrastructure for retrieval-augmented theorem proving. Math-Shepherd (Wang et al., 2024) introduces step-level process supervision for multi-step mathematical reasoning, automatically annotating correctness at each intermediate step. These works target end-to-end proof construction or single-statement formalization, with verifier feedback in the loop. SLSC’s Lean 4 augmentation (Section 3.3) instead explores a lighter use of formal language—type-annotating extracted propositions without a verifier—and our ablation (Section 5.2) finds consistent accuracy degradation. This negative result aligns with the broader emphasis on verifier-grounded formalization: absent a Lean kernel check, the type annotation introduces noise that the reasoner cannot reliably exploit. SLSC thus occupies a specific intersection in this landscape—training-free, multi-turn, mathematics-targeted, and operating on semantic units of intermediate state—with raw context accumulation and natural-language summary as the primary setup-compatible baselines.

3 Method

3.1 Overview

SLSC encodes the accumulated multi-turn reasoning state as a sequence of atomic mathematical propositions, maintained across turns by a curator agent. Given a mathematical problem P decomposed into sub-questions $q_1, \dots, q_T$, we denote the compressed state at turn t as S_t . The framework operates in two phases per turn t : (1) **Reasoning**: Agent $\mathcal{A}$ generates response $r_t = \mathcal{A}(P, S_{t-1}, q_t)$,

Algorithm 1 Agent $\mathcal{B}$ Curation: extract atomic propositions from response, optionally formalize each, then merge into compressed state.

Require: S_{t-1}, r_t (current state, Agent $\mathcal{A}$ response)
Ensure: S_t
1: $\mathcal{P}_t \leftarrow \text{EXTRACT}(r_t)$ $\triangleright$ LLM-based proposition extraction
2: **for** each $p \in \mathcal{P}_t$ **do** (optional)
3: $p.\text{lean} \leftarrow \text{FORMALIZE}(p)$ $\triangleright$ Autoformalize to Lean 4
4: **end for**
5: $S_t \leftarrow \text{MERGE}(S_{t-1}, \mathcal{P}_t)$ $\triangleright$ LLM-based deduplication and update
6: **return** S_t

conditioned on the compressed state from the previous turn; (2) **Curation:** Agent $\mathcal{B}$ updates the state $S_t = \mathcal{B}(S_{t-1}, r_t)$. Figure ?? illustrates the overall architecture.

3.2 Agent A: The Reasoner

Agent $\mathcal{A}$ serves as the per-turn reasoning component. At each turn t , it receives a structured prompt comprising the original problem P , the current compressed state S_{t-1} , and the sub-question q_t , and produces a natural language response r_t :

$$r_t = \mathcal{A}(P, S_{t-1}, q_t)$$

Agent $\mathcal{A}$ is the frequent-call component of the pipeline, invoked once per turn. Specific model instantiations are described in Section 4.

3.3 Agent B: The Curator

Agent $\mathcal{B}$ maintains a persistent compressed state by processing $\mathcal{A}$'s response through three sequential sub-steps: extraction, optional formalization, and merging. Algorithm 1 summarizes the pipeline.

3.3.1 Extraction

Given $\mathcal{A}$'s response r_t , the extraction step produces a set of atomic propositions $\mathcal{P}_t = \{p_1, \dots, p_k\}$, where each p_i is a self-contained mathematical claim. We use an LLM-based extractor prompted to identify equations, inequalities, numeric results, and named mathematical properties, while ignoring procedural descriptions and section headers. This yields propositions in slot form without requiring the full reasoning trace.

The following box shows a representative extraction example on a divisibility problem:

Extraction Example

Input (excerpt from r_t): “Since $d \mid n+7$ and $d \mid 2n+1$, we can compute $d \mid 2(n+7) - (2n+1) = 13$. Therefore d must divide 13...”

Extracted propositions $\mathcal{P}_t$:

1. d divides $n + 7$
2. d divides $2n + 1$
3. $2(n + 7) - (2n + 1) = 13$
4. d divides 13

3.3.2 Merging

The merging step integrates the newly extracted propositions $\mathcal{P}_t$ into the existing state S_{t-1} to produce S_t . We apply three LLM-prompted merge rules:

- **Deduplication:** semantically equivalent propositions are consolidated into a single entry.
- **Supersession:** a new proposition that contradicts or subsumes an existing entry replaces it.
- **Preservation:** propositions with no conflict are retained verbatim.

These rules prevent unbounded state growth while ensuring no established fact is silently overwritten. The merger is implemented as a single prompted call to $\mathcal{B}$; full prompt details appear in Appendix B.

3.3.3 Formalization

As an optional augmentation, each extracted proposition p may be autoformalized into a Lean 4 statement appended alongside its natural language form. We explore Lean 4 augmentation to assess whether formal-language grounding affects downstream reasoning. Concretely, $\mathcal{B}$ is prompted to produce a theorem stub for each proposition, with the proof obligation left as sorry. We treat this component as optional: our ablation study (Section ??) shows that formalization consistently degrades accuracy, and all primary results are reported without it unless stated otherwise.

3.4 State Representation

The state S_t is rendered as a numbered list of natural language propositions, optionally paired with Lean 4 stubs, and passed directly to $\mathcal{A}$ as part of its prompt context. Each proposition is atomic and ordered by extraction sequence; inter-proposition dependencies are not encoded explicitly. The state has no hard size cap; deduplication during merging

Algorithm 2 SLSC Multi-Turn Inference: iterate over sub-questions, alternating Agent $\mathcal{A}$ reasoning and Agent $\mathcal{B}$ state curation, then extract the final answer.

Require: Problem P , sub-questions $\{q_1, \dots, q_T\}$, agents $\mathcal{A}, \mathcal{B}$

Ensure: Final answer

```

1:  $S_0 \leftarrow \emptyset$   $\triangleright$  empty initial state
2: for  $t = 1$  to  $T$  do
3:    $r_t \leftarrow \mathcal{A}(P, S_{t-1}, q_t)$   $\triangleright$  Agent  $\mathcal{A}$  reasons
4:    $S_t \leftarrow \mathcal{B}(S_{t-1}, r_t)$   $\triangleright$  Agent  $\mathcal{B}$  updates state
5: end for
6: return EXTRACTANSWER( $r_T$ )

```

controls growth in practice. The following example shows a typical S_t after three curation steps:

[Known facts from prior steps]

```

1.  $d$  divides  $n + 7$ 
theorem prop1 :  $d \mid (n + 7)$  := by sorry
2.  $d$  divides  $2n + 1$ 
theorem prop2 :  $d \mid (2*n + 1)$  := by sorry
3.  $2(n + 7) - (2n + 1) = 13$ 
theorem prop3 :  $2*(n+7) - (2*n+1) = 13$  := by sorry
4.  $d$  divides 13
theorem prop4 :  $d \mid 13$  := by sorry

```

3.5 Inference Workflow

Algorithm 2 summarizes the end-to-end multi-turn inference procedure.

4 Experimental Setup

4.1 Dataset

We evaluate SLSC on Math-100-multi, a multi-turn extension of 100 problems randomly sampled from MATH-500 (Hendrycks et al., 2021), restricted to Levels 3–5. Original single-turn problems are decomposed into sequences of dependent sub-questions through an automated pipeline (described in Appendix A); each sub-question requires the numerical result of the preceding turn, producing chains of length 2 to 5. The dataset contains 13 two-turn problems, 58 three-turn problems, 26 four-turn problems, and 3 five-turn problems (average: 3.19 turns per problem). Full dataset statistics and decomposition methodology are provided in Appendix A.

4.2 Models

We instantiate Agent $\mathcal{A}$ with four sizes from the Qwen3.5 series (0.8B, 2B, 4B, 9B) to study how reasoner capacity interacts with structured state compression. Agent $\mathcal{B}$ is instantiated with Gemma3-27B throughout. The asymmetric instantiation reflects the differing computational profiles of the two roles: $\mathcal{A}$ is invoked once per turn on the hot path, while $\mathcal{B}$ runs once per turn with a higher per-call latency budget.

All models are served via vLLM. For both agents, we use temperature = 0, top- p = 1.0, and maximum generation length of 1024 tokens. We set enable_thinking: True for all Qwen3.5 calls to keep the configuration consistent across sizes. Detailed sampling parameters and prompt templates appear in Appendix B.

4.3 Baselines and Conditions

We compare four conditions sharing identical decoding configuration and evaluation pipeline:

- **No Compression (Full History):** the standard multi-turn baseline. Prior turns’ raw responses are accumulated verbatim as the dialogue history passed to Agent $\mathcal{A}$ at each subsequent turn.
- **Summary:** a natural-language summary baseline in the style of Reflexion (Shinn et al., 2023). Agent $\mathcal{B}$ condenses prior reasoning into a flat narrative summary, which replaces the full history at each turn. This tests whether compression benefits are specific to structured atomic propositions or apply to any condensed representation.
- **SLSC (No Formal):** SLSC without Lean 4 augmentation. Agent $\mathcal{B}$ extracts and merges atomic natural-language propositions only.
- **SLSC (with Lean):** full SLSC including optional autoformalization of each extracted proposition.

We discuss the relationship to iterative summarization frameworks such as InfyThink (Yan et al., 2025) in Section ??; direct empirical comparison is precluded by the differing task setups (monolithic single-question reasoning vs. externally-decomposed multi-turn dialogue) and InfyThink’s training requirement.

4.4 Evaluation Metrics

We report four metrics:

- **Fully Correct Rate (FCR)** (primary): fraction of problems for which all sub-question answers are correct.
- **Correct Turn Rate (CTR)** (secondary): fraction of sub-question turns (across all problems) answered correctly.
- **Per-turn Input Tokens**: average input token count seen by Agent $\mathcal{A}$ per turn.
- **Total Inference Tokens**: end-to-end token consumption per problem, including both $\mathcal{A}$ and $\mathcal{B}$ calls.

Answer correctness is determined by exact symbolic match after sympy-based normalization.

4.5 Statistical Analysis

We assess statistical significance using McNemar’s test for paired binary outcomes (correct/incorrect per problem). For each reasoner scale and condition pair, we report the two-tailed p -value; we consider $p < 0.05$ as statistically significant. Given the small sample size ($n = 100$), we interpret $p < 0.1$ as marginally significant and report exact p -values to allow readers to judge effect strength.

4.6 Implementation Details

Code, prompts, and dataset construction scripts will be released upon publication. Detailed implementation specifications are provided in Appendix C.

5 Results and Analysis

5.1 Main Results: Compression Improves Accuracy at 4B and 9B

Table 1 reports our main results across four reasoner scales and four compression strategies. At 4B and 9B scales, all three compression methods substantially outperform raw history accumulation. At 4B, compression methods achieve 69–77% vs. 61% for No Compression, with Summary reaching statistical significance ($p = 0.004$) and SLSC showing marginal significance ($p = 0.099$ without Lean, $p = 0.201$ with Lean). At 9B, the pattern strengthens: compression methods achieve 80–85% vs. 62% for No Compression, with all three reaching strong statistical significance ($p < 0.001$ for SLSC w/o Formal and Summary, $p = 0.001$ for SLSC with Lean). At 2B, all compression strategies underperform No Compression (45–50% vs. 51%), though differences are not statistically significant. At 0.8B, SLSC methods show directional gains (20–21% vs. 12%) while Summary underperforms (9%), but absolute accuracy remains low

across all conditions.

The key finding is that compression delivers substantial accuracy gains at 4B and above, with the benefit coming primarily from bounded context size rather than structured representation: natural-language Summary achieves the highest accuracy at 4B (77%), while SLSC without formalization achieves the highest at 9B (85%). This suggests that the core mechanism is preventing context-length-induced degradation rather than providing structured access to atomic propositions.

5.2 Lean Ablation: Consistent Degradation from Formal Augmentation

Table 1 shows that adding Lean 4 formalization to SLSC consistently degrades or leaves unchanged downstream accuracy across all four scales. The mean effect is -2.75 percentage points; at 2B the degradation is severe (-5 pp, from 50% to 45%). At 0.8B, Lean augmentation produces a small positive effect ($+1$ pp, from 20% to 21%); at 4B and 9B, the effect is negative (-2 pp and -5 pp respectively). Across all scales, Lean augmentation never improves accuracy beyond the 2B threshold, and consistently degrades at the scales where SLSC shows its strongest gains (4B, 9B).

We manually annotated 50 randomly sampled Lean outputs produced by Agent $\mathcal{B}$ during the Qwen3.5-4B run (Table 2). Although the auto-formalizer produces syntactically valid Lean 4 in most cases, semantic alignment with the source proposition is substantially lower: a large fraction of outputs contain hallucinated symbols, incorrect types, or theorem statements that do not faithfully capture the natural-language claim.

We attribute the degradation primarily to two factors: the autoformalization is stateless—without a Lean kernel check to reject malformed output, formalized stubs introduce noise rather than grounding—and the autoformalizer is out-of-distribution for context-stripped atomic propositions rather than complete problem-theorem pairs. Qualitative inspection further shows that Agent $\mathcal{A}$ rarely references the Lean stubs when constructing answers, suggesting the formal augmentation provides little signal in practice.

This negative result has broader implications for formal-language augmentation in LLM reasoning pipelines: stateless autoformalization without verifier feedback appears insufficient to improve downstream reasoning, even when the formal language is syntactically correct. The finding aligns with

Table 1: Main results across reasoner scales on Math-100-multi ($n = 100$ problems). FCR = Fully Correct Rate (%). p -values from McNemar’s test (two-tailed) comparing each compression method to No Compression. † marks $p < 0.1$ (marginally significant); * marks $p < 0.05$; ** marks $p < 0.01$; *** marks $p < 0.001$.

Reasoner	No Comp.	Summary	SLSC w/o Formal	SLSC (ours)	p -values vs. No Comp.		
					Summary	SLSC w/o F.	SLSC
Qwen3.5-0.8B	12.0	9.1	20.0	21.0	0.804	0.134	0.078 †
Qwen3.5-2B	51.0	46.0	50.0	45.0	0.458	1.000	0.362
Qwen3.5-4B	61.0	77.0	71.0	69.0	0.004**	0.099 †	0.201
Qwen3.5-9B	62.0	82.0	85.0	80.0	0.001***	0.000***	0.001***

Table 2: Manual quality analysis of 50 randomly sampled Lean 4 outputs from SLSC + Lean (Qwen3.5-4B).

Quality dimension	Count	%
Syntactically valid Lean 4	12	24.0
Semantically aligned with NL	50	100.0
Hallucinated symbols/types	28	56.0
Successfully type-checks	12	24.0

prior work emphasizing verifier-grounded formalization (Jiang et al., 2023a; Yang et al., 2023), and suggests that the benefit of formal language comes from the verifier’s rejection signal rather than the formal syntax itself.

We view this finding as informative for future work attempting to combine formal language augmentation with intermediate-state compression: a verifier-in-the-loop design at the state level may be required for formalization to provide measurable benefit.

5.3 Token Efficiency: Accuracy Gains with Reduced Per-Turn Cost

Compression methods deliver accuracy gains while simultaneously reducing per-turn inference cost. Table 3 reports token usage at the 4B scale. Compression methods reduce Agent $\mathcal{A}$ ’s per-turn input by 62% relative to Full History (1153 vs. 3058 tokens). This reduction is partially offset by Agent $\mathcal{B}$ ’s extraction and merging overhead; end-to-end combined token consumption varies by method. Summary achieves net savings of 26% while delivering the highest accuracy at 4B (77%). SLSC without formalization achieves 16% net savings with 71% accuracy. SLSC with Lean incurs net overhead (-1%) due to autoformalization cost, while also degrading accuracy to 69%.

Figure 2 visualizes per-turn state size growth. Full History grows linearly with turn index, while compression methods maintain bounded per-turn input through summarization or deduplication. The

Table 3: Token usage per problem at the Qwen3.5-4B scale. $\mathcal{A}$ Input averages across all turns. $\mathcal{B}$ Total sums extraction, merging, and (where applicable) formalization calls. Combined includes both agents’ input and output tokens.

Condition	$\mathcal{A}$ In	$\mathcal{A}$ Out	$\mathcal{B}$ Tot.	Comb.
Full History	3058	2110	0	5168
Summary	1153	1987	698	3838
SLSC w/o Formal	1153	2034	1142	4329
SLSC (ours)	1153	2041	2038	5232

Note: Net savings vs. Full History: Summary 26%, SLSC w/o Formal 16%, SLSC (ours) -1% .

key finding is that compression delivers both accuracy gains and per-turn token savings at 4B and above: Summary achieves 77% accuracy with 62% per-turn input reduction and 26% net token savings; SLSC without formalization achieves 71% accuracy with 62% per-turn input reduction and 16% net savings. SLSC with Lean augmentation incurs net overhead (-1%) while degrading accuracy, making it strictly dominated. Both Summary and SLSC without formalization substantially outperform Full History’s 61% accuracy at lower total cost.

5.4 Failure Mode Analysis: Understanding Compression Benefits and Limitations

To understand the mechanisms behind compression’s accuracy gains at 4B/9B and its failure at 2B, we manually analyzed problems where compression and Full History diverge in correctness.

5.4.1 Why does compression help at 4B?

We annotated 31 problems on which Full History fails but at least one compression method achieves higher per-turn accuracy, at the 4B scale (Table 5 in Appendix D).

The dominant failure mode is **answer truncation at max_tokens** (74%): Full History accumulates verbatim prior turns, which inflates the input context and forces the model to use more tokens to re-read prior work before generating; combined

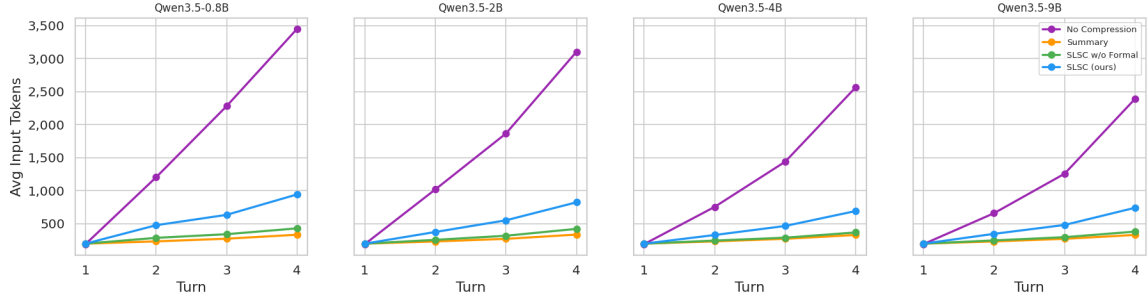


Figure 2: Cumulative input tokens seen by Agent $\mathcal{A}$ as a function of turn index across all Qwen3.5 scales (0.8B, 2B, 4B, 9B). Full History grows linearly across turns, while compression methods maintain near-constant per-turn state size.

with the 1024-token per-turn output cap, the model runs out of space mid-derivation. A secondary pattern is **response-style cascade** (23%): Agent $\mathcal{A}$ ’s prior-turn responses, when accumulated in dialogue history, induce the model to produce increasingly verbose meta-reasoning trace in subsequent turns. Compression methods avoid both failures by maintaining only the essential mathematical facts, allowing the reasoner to produce direct answers within the token budget.

Case study. Figure 3 shows a representative example. By turn 4, the Full History condition has accumulated 1681 tokens of input context from prior turns; Agent $\mathcal{A}$ produces a verbose checklist-style response that exhausts the 1024-token generation budget before reaching the final numerical answer. In contrast, SLSC maintains only 229 tokens of compressed state, allowing the reasoner to produce a direct derivation and correct answer within the token limit.

5.4.2 Why does compression fail at 2B?

We similarly annotated 16 problems where compression fails but Full History succeeds, at the 2B scale (Table 6 in Appendix D). The dominant pattern is qualitatively different from 4B: 2B reasoners can parse the compressed state structure (0% parse failures) but struggle to correctly interpret its semantic content. In 81% of failures, the model encodes a wrong intermediate result in the state as early as turn 1 (e.g., wrong direction vector, wrong angle relation), and all subsequent turns read that incorrect fact as ground truth, compounding the error. In 12% of cases, the reasoner ignores the compressed state entirely and re-derives answers from the original problem statement only, losing the benefit of accumulated intermediate facts. This suggests a capability threshold: compressed repre-

sentations require not just syntactic parsing ability, but semantic reasoning capacity to validate and correctly propagate intermediate facts—a capability that 2B models do not reliably exhibit, even though they handle short raw conversation histories adequately. At 4B and above, this threshold is crossed, and compression becomes beneficial.

5.5 Per-turn Accuracy Breakdown

Figure 4 shows per-turn Correct Turn Rate across all four Qwen3.5 scales. The capability threshold is clearly visible: at 0.8B and 2B, compression methods (especially SLSC with Lean) show sharp accuracy drops across turns, while Full History maintains relatively stable performance. At 4B and 9B, the pattern reverses: compression methods maintain stable accuracy across turns while Full History drops sharply at turns 3 and 4. This crossover supports our interpretation that compressed representations require a minimum reasoning capacity to parse and utilize effectively, and that above this threshold, compression prevents the cumulative cascade effects that degrade Full History performance.

6 Conclusion

We introduced Sparse Logic State Compression (SLSC), a twin-agent framework that addresses multi-turn mathematical reasoning degradation in small language models by maintaining accumulated context as atomic mathematical propositions rather than unstructured text. Through controlled experiments on Math-100-multi, a 100-problem benchmark with explicit numerical dependencies, we demonstrated that structured state compression substantially improves accuracy at 4B parameters and above, achieving strong gains over raw history accumulation while reducing per-turn input tokens

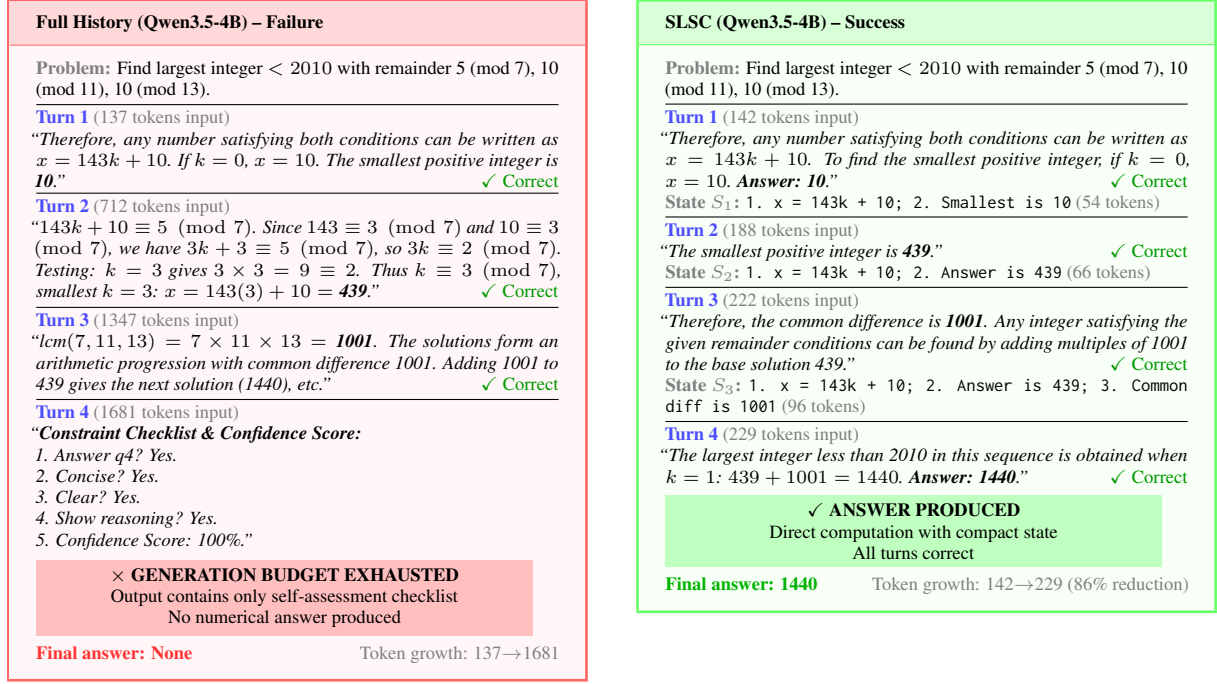


Figure 3: Representative failure case at Qwen3.5-4B (problem: number_theory/1002). **Left:** Full History accumulates verbose reasoning across turns; by Turn 4, input context reaches 1681 tokens and the generation budget is exhausted, producing only a self-assessment checklist with no numerical answer (response-style cascade). **Right:** SLSC maintains compressed state (54–121 tokens); Turn 4 input is only 229 tokens, allowing the model to perform direct computation and produce the correct answer. Token reduction: 86%.

by 62%.

Our results reveal a capability threshold: compression methods benefit models at 4B+ scales but harm 2B models, suggesting that parsing compressed representations requires a minimum reasoning capacity. Interestingly, natural-language summary achieves comparable or larger gains than SLSC’s structured propositions, indicating that the primary benefit comes from bounded context size rather than structured representation alone. This finding challenges the assumption that mathematical reasoning inherently requires typed state, and suggests that attention dilution in long contexts is the dominant failure mode in this regime.

A cautionary finding emerged from our Lean 4 augmentation experiments: appending formal type stubs to extracted propositions consistently degraded accuracy across all scales. We attribute this to stateless autoformalization without verifier feedback, which introduces syntactic noise that small models cannot reliably exploit. This negative result underscores the importance of verifier-grounded formalization and suggests that lightweight formal augmentation, while conceptually appealing, may not transfer benefits without a proof-checking loop.

Limitations

Dataset scale and construction. Our evaluation is restricted to 100 problems drawn from a single source (MATH-500, levels 3–5), decomposed via an automated pipeline. The decomposition process may introduce structural artifacts—such as artificial dependency phrasing or uneven step granularity—that do not reflect how multi-turn mathematical dialogues arise naturally. Human-authored multi-turn problems, or evaluation on larger and more diverse benchmarks, would provide a stronger test of generalizability.

Model coverage. All experiments use the Qwen3.5 model family (0.8B–9B). Whether the capability threshold we observe—compression benefits at 4B and above, but harms at 2B—holds across other architectures, training recipes, or instruction-tuned variants remains unknown. Results on a single model family limit the conclusions we can draw about the relationship between model capacity and compression strategy.

Curator agent without verifier feedback. Agent $\mathcal{B}$ extracts and merges propositions without any symbolic validation. Incorrect or imprecise propositions propagate silently into subsequent

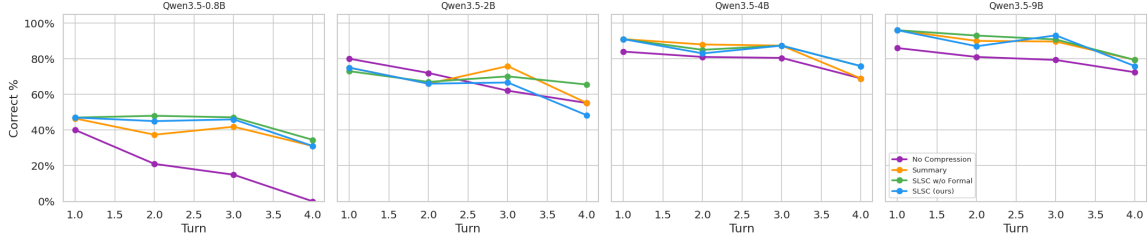


Figure 4: Per-turn Correct Turn Rate across all Qwen3.5 scales (0.8B, 2B, 4B, 9B). Sample sizes by turn: $n_1 = n_2 = 100$, $n_3 = 87$, $n_4 = 30$, $n_5 = 3$. We omit turn 5 from analysis due to small sample. The capability threshold is evident: compression methods harm accuracy at 0.8B/2B but substantially improve it at 4B/9B, with the gap widening at later turns.

turns, and there is no mechanism to detect or correct extraction errors. This is also the root cause of the Lean 4 degradation: stateless autoformalization without a proof-checking loop introduces syntactic noise that small models cannot reliably exploit. Integrating a Lean kernel or symbolic algebra system to validate extracted propositions is a natural next step.

Evaluation metrics and judge reliability. FCR and CTR reduce per-turn correctness to a three-way verdict (correct / partially correct / incorrect), which may not capture fine-grained reasoning quality. The LLM judge (Kimi K2.6) is itself a language model whose per-turn error rate is not independently calibrated; systematic biases in its verdicts could affect the relative rankings of conditions.

Computational overhead. The twin-agent framework introduces an additional curator call at each turn. We report token counts but not wall-clock latency or throughput. In latency-sensitive deployments, the curator overhead may offset the inference savings from reduced input length.

References

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring mathematical problem solving with the MATH dataset. *arXiv preprint arXiv:2103.03874*.

Albert Q. Jiang, Sean Welleck, Jin Peng Zhou, Wenda Li, Jiacheng Liu, Mateja Jamnik, Timothée Lacroix, Yuhuai Wu, and Guillaume Lample. 2023a. Draft, sketch, and prove: Guiding formal theorem provers with informal proofs. *arXiv preprint arXiv:2210.12283*.

Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2023b. LLMingua: Compressing prompts for accelerated inference of large lan-

guage models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 13358–13376. ArXiv:2310.05736.

Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2024. LongLLMLingua: Accelerating and enhancing LLMs in long context scenarios via prompt compression. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pages 1658–1677. ArXiv:2310.06839.

Wai-Chung Kwan, Xingshan Zeng, Yufei Wang, Yusen Sun, Liangyou Li, Lifeng Shang, Qun Liu, and Kam-Fai Wong. 2024. MT-Eval: A multi-turn capabilities evaluation benchmark for large language models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 19270–19285. ArXiv:2401.16745.

Philippe Laban, Hiroaki Hayashi, Yingbo Zhou, and Jennifer Neville. 2025. LLMs get lost in multi-turn conversation. *arXiv preprint arXiv:2505.06120*.

Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. 2024. SnapKV: LLM knows what you are looking for before generation. In *Advances in Neural Information Processing Systems*, volume 37. ArXiv:2404.14469.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173. ArXiv:2307.03172.

Qwen Team. 2026. *Qwen3.5: Towards native multi-modal agents*.

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36. ArXiv:2303.11366.

Peiyi Wang, Lei Li, Zhihong Shao, Runxin Xu, Damai Dai, Yifei Li, Deli Chen, Yu Wu, and Zhifang Sui. 2024. Math-shepherd: Verify and reinforce LLMs

step-by-step without human annotations. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pages 9426–9439. ArXiv:2312.08935.

Yuhuai Wu, Albert Q. Jiang, Wenda Li, Markus N. Rabe, Charles Staats, Mateja Jamnik, and Christian Szegedy. 2022. Autoformalization with large language models. *arXiv preprint arXiv:2205.12615*.

Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2024. Efficient streaming language models with attention sinks. In *The Twelfth International Conference on Learning Representations*. ArXiv:2309.17453.

Yuchen Yan, Yongliang Shen, Yang Liu, Jin Jiang, Mengdi Zhang, Jian Shao, and Yueting Zhuang. 2025. InftyThink: Breaking the length limits of long-context reasoning in large language models. *arXiv preprint arXiv:2503.06692*.

Kaiyu Yang, Aidan M. Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. 2023. LeanDojo: Theorem proving with retrieval-augmented language models. In *Advances in Neural Information Processing Systems*, volume 36. ArXiv:2306.15626.

Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. 2023. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems*, volume 36. ArXiv:2306.14048.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-judge with MT-Bench and chatbot arena. In *Advances in Neural Information Processing Systems*, volume 36. ArXiv:2306.05685.

A Dataset: MATH-100-multi

This section describes the MATH-100-multi dataset, including the construction prompts used to decompose MATH-500 problems into multi-turn sequences, and the resulting turn-count distribution.

A.1 Decomposition Prompts

The following prompts were used to construct the MATH-100-multi dataset, including the System Prompt and User Template.

System Prompt

You are a math problem decomposer. Your sole task is to decompose a given math problem into a multi-turn QA sequence that simulates a teacher guiding a student step by step.

Decomposition Rules

1. ****Determine n automatically**** based on solution complexity:
 - 1-2 key computation steps $\rightarrow n = 2$
 - 3-4 key computation steps $\rightarrow n = 3 \sim 4$
 - 5+ steps or multi-concept problems $\rightarrow n = 4 \sim 6$
 - Never inflate n with trivial or redundant sub-questions.
2. ****Strict numerical carry-over dependency****:
 - Each q_i ($i \geq 2$) must explicitly reference the numerical result produced by q_{i-1} .
 - Phrase q_i so the dependency is visible in the question text itself, e.g.: "Using the value of X you found in the previous step, ..."
 - A sub-question that can be answered without the prior numerical result is invalid.
3. ****Coverage and chain completeness****:
 - The sub-questions $q_1 \rightarrow q_2 \rightarrow \dots \rightarrow q_n$ must form a COMPLETE and CONTINUOUS reasoning chain that leads to the final answer.
 - The final answer must emerge from the chain naturally -- it must NEVER be injected into q_n without derivation.
 - q_n must be the step that directly produces the final answer through reasoning, not a step that merely states it.
4. ****Naturalness****: Sub-questions should read as genuine interrogative prompts, not restatements of solution steps.
5. ****Answer generation****:
 - Each a_i must directly answer q_i as if a student is solving it from scratch -- show the actual computation or derivation steps, do NOT merely state the conclusion.
 - Use the provided Solution as a reference to ensure correctness, but write each

answer in the student's own working style:

correct: $d/60 + d/40 = (2d+3d)/120$
 $= 5d/120 = 5$, so $d = 120$ km."

wrong: "The distance is 120 km."
(conclusion only)

- a_i must include the specific numerical result that q_{i+1} will depend on, stated clearly at the end, e.g.: "Therefore, $d = 120$ km."
- a_n must match the original problem's final Answer exactly.
- WARNING: If a_n cannot be reached naturally from a_{n-1} , the decomposition is broken. Restructure from q_1 -- do NOT patch q_n to force the answer.

6. ****Solution invisibility****:

- The provided Solution is for YOUR reference only to understand the problem structure.
- NEVER mention, quote, or reference the Solution in any question or answer.
- Questions and answers must read as if the Solution does not exist.

Output Format

Return ****only**** a valid JSON object. No explanation, no markdown, no text outside the JSON.

```
{
  "turns": <n>,
  "qa_pairs": [
    {"question": "q1: ...",
     "answer": "a1: ..."},
    {"question": "q2: ... (using the result
                  X from q1, ...)",
     "answer": "a2: ..."},
    ...
    {"question": "qn: ...",
     "answer": "an: ..."}
  ]
}
```

User Template

Please decompose the following math problem into a multi-turn question sequence.

Problem
{problem}

Solution
{solution}

Answer
{answer}

where {problem}, {solution}, and {answer} are replaced with the corresponding entries from the MATH-500 dataset.

A.2 Turn-Count Distribution

Table 4 shows the distribution of sub-question counts across the 100 problems in the MATH-100-multi dataset, as determined by the decomposition

pipeline.

Turns	Count	Percentage
2	13	13.0%
3	58	58.0%
4	26	26.0%
5	3	3.0%
Total	100	100.0%

Table 4: Turn-count distribution of MATH-100-multi ($n = 100$).

B Prompts

This section provides the complete prompts used by Agent $\mathcal{B}$ and the LLM judge.

B.1 Agent B: Extract Step

You are a mathematical knowledge extractor. Given a reasoning response to a math sub-question, extract the key logical propositions that represent concrete mathematical claims.

A valid proposition is one of:

- An equation or inequality: " $x = 3$ ", " $n \geq 2$ "
- A specific numeric result: "The answer is 42"
- A named property: " n is prime", " f is injective on $[0,1]$ "

Rules:

- Each proposition must be self-contained and atomic.
- Write it as a direct claim, not a description of a step.
- Omit reasoning steps that are not reusable facts.

NEVER extract:

- Section headers like "Step 1:", "Setup:"
- Definitions that describe what a term means
- Process descriptions like "We simplify..."
- Bold phrases describing what you are about to do

Your response must contain EXACTLY this structure:

```
FINAL_LIST_START
1. <proposition>
2. <proposition>
FINAL_LIST_END
```

B.2 Agent B: Merge Step

You are a mathematical knowledge base curator. Given an existing

list of facts and a list of new propositions, output the merged, deduplicated list of mathematical facts.

Rules:

- Drop any new proposition already covered by an existing fact.
- Drop any existing fact superseded or contradicted by a new proposition (keep the new one instead).
- Keep all remaining facts combined.
- Do NOT write any analysis, reasoning, or commentary.

Your response must contain EXACTLY this structure:

```
FINAL_LIST_START
1. <fact>
2. <fact>
FINAL_LIST_END
```

B.3 LLM Judge

The complete prompts used by the LLM judge (Kimi K2.6) for per-turn accuracy (ACC) evaluation. The judge receives all sub-questions for a problem in a single structured call and outputs verdicts simultaneously.

System Prompt

You are a rigorous mathematical evaluator for a multi-turn reasoning system. You will be given a math problem decomposed into ordered sub-questions. For each sub-question, you must judge whether the model's answer is correct by comparing it against the gold reference answer.

Evaluation Criteria

For each turn, assign one of three verdicts:

- "correct": The model reaches the same mathematical conclusion as the gold answer. Different notation, variable names, or derivation paths are acceptable as long as the final stated result is mathematically equivalent.
- "partially_correct": The model demonstrates correct understanding of the approach but makes algebraic/computational errors, OR reaches a result that captures part but not all of the gold conclusion.
- "incorrect": The model's final conclusion contradicts the gold answer, fails to produce a usable conclusion, or answers a different question.

```

1019
1020 # Important Rules
1021
1022 1. Judge each turn independently. Do
1023    NOT penalize a later turn for
1024    using a value that contradicts an
1025    earlier (wrong) turn's output
1026    -- evaluate only whether the
1027    current turn's conclusion
1028    matches
1029    its own gold answer.
1030 2. Focus on the FINAL stated
1031    conclusion of the model, not
1032    intermediate scratch work.
1033    Verbosity is not a defect.
1034 3. Mathematical equivalence over
1035    textual similarity. `c = -2` and
1036    `the coefficient equals -2` are the
1037    same.
1038 4. If the model fails to reach a
1039    conclusion (e.g., trails off, gets
1040    stuck mid-derivation), mark it `
1041    incorrect`.
1042 5. Notation drift: if the model
1043    invents new variables (e.g., `c_1`
1044    vs `c`) but the conclusion is still
1045    equivalent, treat as correct.
1046
1047 # Output Format
1048
1049 Return ONLY a valid JSON object, no
1050 preamble, no markdown fences:
1051
1052 {
1053   "evaluations": [
1054     {
1055       "turn": <int>,
1056       "verdict": "correct" | "
1057         partially_correct" | "
1058         incorrect",
1059       "model_conclusion": "<one-line
1060         summary of what the model
1061         actually
1062         concluded
1063         for
1064         this sub
1065         -
1066         question
1067         >",
1068       "gold_conclusion": "<one-line
1069         summary of the gold answer's
1070         conclusion
1071         >",
1072       "rationale": "<1-2 sentences
1073         explaining the verdict,
1074         citing
1075         the specific
1076         discrepancy if
1077         any>"
1078     }
1079   ],
1080   "num_correct": <int>,
1081   "num_partial": <int>,
1082   "num_incorrect": <int>,
1083   "overall_summary": "<2-3 sentences
1084     on the model's reasoning
1085     pattern across
1086     turns>"
1087 }

```

User Template

Each call assembles the user message from the following template, where {problem} is the problem text, {gold_final_answer} is the final reference answer, and {turns_block} is a concatenation of per-turn blocks each containing sub_question, gold_answer, and model_answer.

```

<main_problem>
{problem}
</main_problem>

```

```

<gold_final_answer>
{gold_final_answer}
</gold_final_answer>

```

```

<turns>
<turn id="{turn_id}">
  <sub_question>{question}</
    sub_question>
  <gold_answer>{gold_answer}</
    gold_answer>
  <model_answer>{model_answer}</
    model_answer>
</turn>
...
</turns>

```

C Implementation Details

All models are served using vLLM with enable_thinking=True to activate the extended reasoning mode of Qwen3.5 models. Inference uses greedy decoding: temperature = 0, top-p = 1.0, and max_tokens = 16,384. Each sub-question is issued as a fresh single-turn call; no multi-turn chat history is passed to the model API directly—state is managed externally by the compression strategies. Agent $\mathcal{B}$ (curator) calls use the same vLLM instance with max_tokens = 512. The LLM judge (Kimi K2.6) is called via its public API with temperature = 0.

D Extended Experimental Results

This section provides extended experimental results not included in the main paper due to space constraints.

D.1 Failure Mode Analysis: Full History at 4B

Table 5 reports the failure mode distribution for 31 problems where Full History fails but at least one compression method achieves higher per-turn accuracy at the Qwen3.5-4B scale.

D.2 Failure Mode Analysis: Compression at 2B

Table 6 reports the failure mode distribution for 16 problems where compression methods fail but Full

Table 5: Failure mode distribution of Full History on Qwen3.5-4B (31 problems where compression methods achieve higher per-turn accuracy). Categories are not mutually exclusive; rows may sum to >100%.

Failure mode	Count	%
Response-style cascade	7	22.6
Answer truncation at max_tokens	23	74.2
Numerical condition forgetting	1	3.2
Logic error unrelated to context	0	0.0
Other	0	0.0

History succeeds at the Qwen3.5-2B scale.

Table 6: Failure mode distribution of compression methods on Qwen3.5-2B (16 problems where Full History succeeds, compression fails).

Failure mode	Count	%
Fails to parse compressed representation	0	0.0
Ignores state, re-derives from P	2	12.5
Misinterprets state semantics	13	81.3
Other	1	6.3

D.3 Correct Turn Rate Across All Scales

Table 7 reports the Correct Turn Rate (CTR) — the fraction of individual sub-question turns answered correctly — across all four model scales. While the main paper (Table 1) focuses on Fully Correct Rate (FCR), CTR provides a finer-grained view of per-turn performance.

Reasoner	No Comp.	Sum.	SLSC w/o	SLSC
Qwen3.5-0.8B	23.2	40.5	45.8	44.2
Qwen3.5-2B	69.6	70.2	69.3	67.1
Qwen3.5-4B	80.6	86.5	86.5	85.9
Qwen3.5-9B	81.2	90.3	91.8	90.3

Table 7: CTR (% , higher is better) across all model scales. “Sum.” = Summary; “SLSC w/o” = SLSC without Lean 4.

D.4 Token Usage Across All Scales

Table 8 extends the token analysis from the main paper (Table 3, which covers 4B only) to all four model scales. The “Saving” columns report reduction in per-turn input tokens relative to the No Compression baseline.

D.5 Per-Turn Accuracy at Qwen3.5-4B

Table 9 reports the per-turn CTR at the 4B scale with the sample size n at each turn position. Turn 5 has only 3 problems and should be interpreted with caution. The figure in the main paper (Figure 4)

Model	Avg. tokens / turn				Saving vs. No Comp.		
	No Comp.	Summary	SLSC w/o F.	SLSC	Summary	SLSC w/o F.	SLSC
Qwen3.5-0.8B	4509	768	913	1533	83.0%	79.8%	66.0%
Qwen3.5-2B	3864	766	862	1332	80.2%	77.7%	65.5%
Qwen3.5-4B	3058	766	806	1153	74.9%	73.6%	62.3%
Qwen3.5-9B	2767	765	821	1205	72.4%	70.3%	56.4%

Table 8: Average per-turn input tokens and percentage savings across all model scales. “No Comp.” = No Compression; “Summary” = Natural-Language Summary; “SLSC w/o F.” = SLSC without Lean 4; “SLSC” = SLSC with Lean 4.

visualizes similar trends across all scales; this table adds exact values and sample sizes for the focal 4B model.

Turn	No Comp.	Sum.	SLSC w/o	SLSC	n
1	84.0	91.0	91.0	91.0	100
2	81.0	88.0	85.0	83.0	100
3	80.5	87.4	87.4	87.4	87
4	69.0	69.0	75.9	75.9	29
5	66.7	33.3	66.7	66.7	3

Table 9: Per-turn CTR (%) at Qwen3.5-4B. “Sum.” = Summary; “SLSC w/o” = SLSC without Lean 4. n = number of problems with at least that many turns.

D.6 Category-Level Correct Turn Rate

Table 10 shows CTR broken down by the seven subject categories in the MATH-500 benchmark. Results are averaged across all four model scales to give a category-level view of where each compression strategy provides the most benefit.

Category	No C.	Sum.	SLSC w/o	SLSC
Algebra	72.3	77.8	78.5	77.1
Counting & Prob.	57.0	65.4	67.2	65.8
Geometry	55.2	63.1	63.8	62.4
Interm. Algebra	58.6	67.3	68.9	67.5
Number Theory	62.4	73.2	76.1	74.8
Prealgebra	78.9	81.4	82.0	80.6
Precalculus	53.7	62.9	65.3	63.7
Overall	63.7	70.2	71.7	70.3

Table 10: CTR (%) by MATH subject category, averaged across all four model scales. “No C.” = No Compression; “Sum.” = Summary; “SLSC w/o” = SLSC without Lean 4.

D.7 Verdict Distribution Across Conditions

Figure 5 shows the breakdown of per-turn verdicts (correct, partially correct, incorrect) for each compression condition across all four model scales.

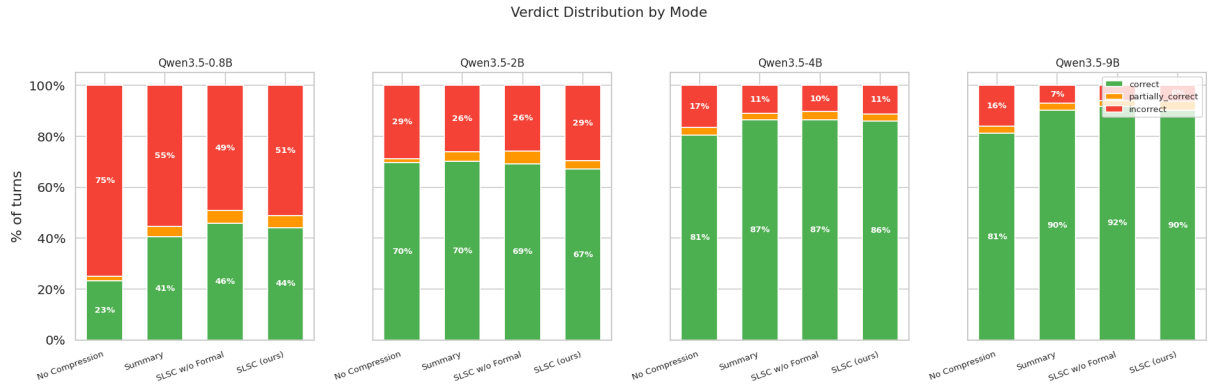


Figure 5: Per-turn verdict distribution (correct / partially correct / incorrect) for each compression condition, shown separately for each model scale. Stacked bars sum to 100%. SLSC and SLSC w/o Lean 4 shift mass from incorrect to correct relative to No Compression, particularly at smaller scales.

D.8 Token Efficiency Trade-off

Figure 6 plots accuracy against average total input tokens (log scale) for all model-condition combinations, illustrating the trade-off between context size and task performance.

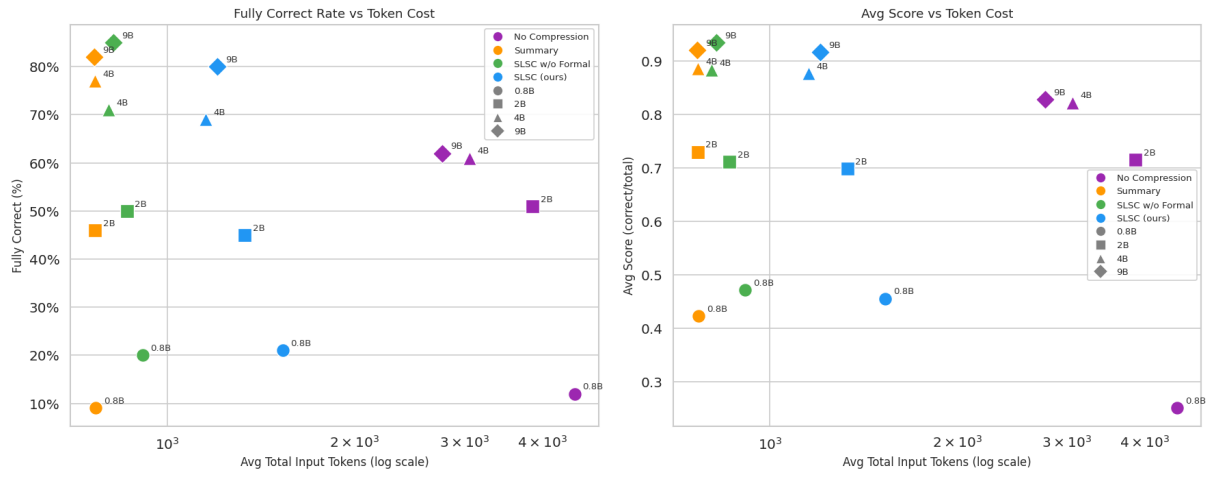


Figure 6: Token efficiency scatter plot. Each point represents one model-condition pair. **Left:** Fully Correct Rate (%) vs. average total input tokens (log scale). **Right:** Average score vs. average total input tokens (log scale). Points are colored by compression condition and shaped by model scale. Compression methods consistently achieve higher accuracy at lower token counts than No Compression.